ReasonFlow: Flow Matching that Thinks Step by Step

Anonymous ACL submission

Abstract

Flow matching has shown strong promise as a continuous generative framework, but its standard formulation is not well-suited to reasoning problems. Conventional flow matching trains a model to transport noise directly toward a single final target. For tasks that require multi-step reasoning, this direct path is often too difficult: the final answer may be reachable only through a sequence of intermediate computations or transformations. Inspired by how autoregressive language models address this challenge through chain-of-thought supervision, we introduce ReasonFlow, a framework that brings the same scaffolding principle to flow matching. Instead of supervising the flow with only a final target, ReasonFlow constructs a sequence of CoT-induced anchor points corresponding to intermediate reasoning states. During training, the clean target moves along a piecewise-linear path through these anchors, so the model learns to progress through reasoning waypoints before reaching the final answer. This turns reasoning supervision from an output sequence into a trajectory-shaping signal for flow matching. Across arithmetic, chemistry, and logical-reasoning tasks, ReasonFlow substantially improves over no-CoT flow-matching baselines.

1 Introduction

Many reasoning problems are difficult to solve by directly predicting the final answer. Solving an arithmetic word problem often requires computing several intermediate quantities before the answer becomes available. Predicting a retrosynthetic route requires decomposing a target molecule through a sequence of plausible precursors. Inferring a reaction mechanism requires tracking a chain of electron-flow operations, and logical deduction requires applying rules over several hops. In all these cases, the final answer is the endpoint of a structured process. Training a model to map the input directly to that endpoint provides only a sparse signal: it tells the model where it should arrive, but not how to get there.

Autoregressive language models address this difficulty through chain-of-thought supervision. Instead of predicting only the final answer, the model is trained to produce intermediate reasoning steps before the answer. These steps act as a scaffold: they break a difficult global prediction into a sequence of easier local transitions, expose useful intermediate states, and provide supervision along the path from problem to solution.

Continuous generative models such as flow matching appear, at first glance, to have a natural mechanism for such step-by-step construction. A flow model starts from noise and progressively transforms it into a final sample through a continuous trajectory. It is tempting to imagine that, on reasoning tasks, this progressive denoising process could itself become a progressive reasoning process: early states might represent partial understanding of the problem, intermediate states might correspond to partial computations or subgoals, and the final state might encode the answer. However, standard flow matching does not impose this structure. The model is typically trained with a single clean target, so every intermediate state is supervised only by its relation to the same final endpoint. The trajectory may be continuous in time, but it is not semantically aligned with reasoning progress.

This suggests a natural question: can we use chain-of-thought supervision to align the progressive denoising process of flow matching with the progressive structure of reasoning?

We introduce ReasonFlow, a flow-matching framework that answers this question through anchored trajectory shaping. Instead of supervising the flow with a single fixed target, ReasonFlow defines a sequence of reasoning anchors induced by chain-of-thought steps. These anchors represent intermediate stages of the solution process. During training, the target becomes time-varying: as

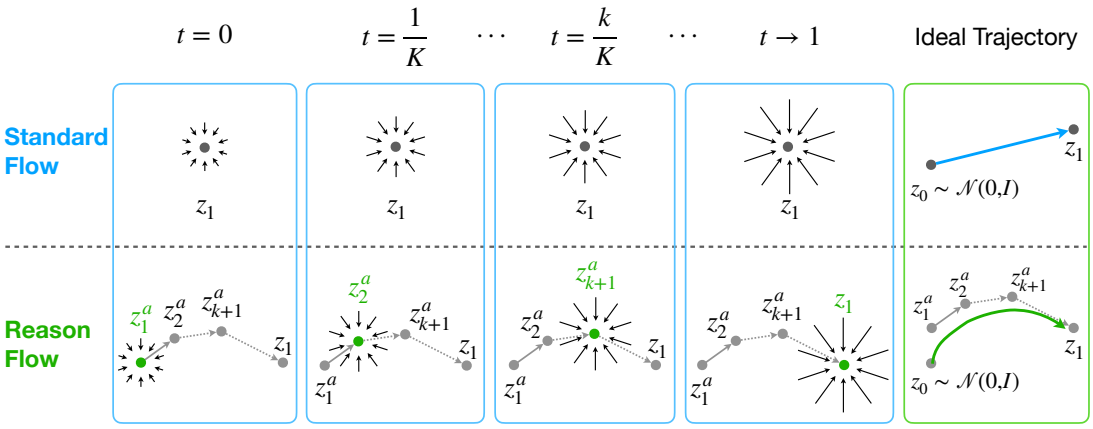

Figure 1: StandardFlow vs. ReasonFlow. Upper left: Standard flow matching trains every intermediate state z_t to predict the same final target z_1, so the vector field points toward the final answer latent at all times. Upper right: Its ideal trajectory directly transports noise z_0 to z_1. Lower left: ReasonFlow makes the clean target time-dependent using reasoning anchors $z_1^a, \ldots, z_K^a$. At $t = k/K$, the target is z_{k+1}^a; between adjacent anchors, it is linearly interpolated. Lower right: The trajectory is pulled toward successive reasoning anchors as time progresses: early denoising steps are guided toward early reasoning states, later steps toward later reasoning states, and the endpoint toward the final answer latent. This aligns the denoising trajectory with the step-by-step structure of reasoning.

the flow time progresses, the target moves along a piecewise-linear path through the anchors. Thus, early denoising times are trained toward early reasoning states, later times are trained toward later reasoning states, and the endpoint corresponds to the final answer. In this way, ReasonFlow turns the denoising trajectory into a scaffolded reasoning trajectory.

This formulation is general: ReasonFlow does not depend on a unique way of constructing the anchors. Any mechanism that maps intermediate reasoning prefixes into continuous representations can, in principle, define the anchor path. In this work, we instantiate this idea using a finetuned causal language model: given a sequence containing the input, chain-of-thought steps, and final answer, we mean-pool hidden-state representations up to reasoning-step boundaries and use them as continuous anchors. Other anchor encoders, representation spaces, or interpolation schemes could be substituted within the same framework.

Importantly, ReasonFlow does not turn flow matching into an autoregressive chain-of-thought generator. The reasoning chain is not generated token by token at inference time, nor is it consumed step by step by the model. Instead, chain-of-thought is only used during training to shape the geometry of the continuous trajectory. At inference, ReasonFlow follows the same flow-based generation process as a no-CoT flow model and decodes the final answer from the endpoint state.

We evaluate ReasonFlow on arithmetic, chemistry, and logical-reasoning tasks. Across these settings, anchored trajectory shaping improves over no-CoT flow-matching baselines, with especially large gains on tasks where direct answer prediction provides a poor learning signal. We further analyze the learned anchor trajectories and show that intermediate flow states can contain partially decodable reasoning information. Overall, our results show that flow matching can benefit from the same scaffolding principle that makes chain-of-thought effective in autoregressive models: difficult reasoning problems become easier when the model is guided through meaningful intermediate states rather than trained only against the final answer.

Code and data for ReasonFlow are available at: `anonymous.4open.science/r/ReasonFlow/`.

2 Latent-Space Rectified Flow

Let x denote the conditioning input and y denote the target output. Before training the flow model, we first learn a continuous latent representation for the target output using an autoencoding objective. An encoder maps the input-output pair (x, y) to a latent vector, which is then normalized to lie on a sphere of radius $\sqrt{d}$:

$$z_1 = \mathrm{sphere}(E_\phi(x, y)), \quad \mathrm{sphere}(z) = \sqrt{d}\frac{z}{\|z\|_2}.$$

A decoder is trained to reconstruct the target output from this latent representation:

$$\hat{y} = D_\psi(z_1).$$

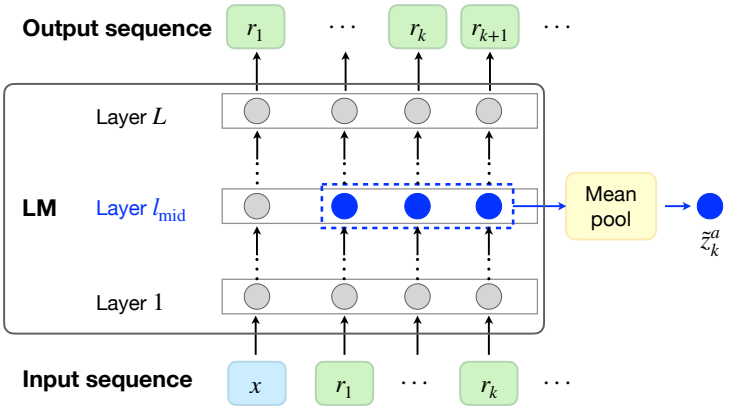

Figure 2: LM-based anchor construction for ReasonFlow. As one concrete instantiation of ReasonFlow, we construct reasoning anchors using a causal transformer language model. The LM reads the sequence $(x, r_1, \ldots, r_K, y)$. For each step k, we mean-pool the middle-layer hidden states over the prefix up to r_k, then project and sphere-normalize the result to obtain anchor z_k^a. This LM-based construction is one concrete instantiation of ReasonFlow while the framework itself only requires a sequence of continuous reasoning anchors.

Thus, z_1 is trained to contain sufficient information to recover the answer y. After this stage, the encoder and decoder define the latent space in which the flow model operates: the flow is trained to generate z_1 from noise, and the decoder maps the generated latent back to the output space.

Given the target latent z_1, latent-space rectified flow learns a conditional transport from a simple base distribution to the distribution of target latents. For each training example, we sample an initial noise vector:

$$z_0 \sim \mathcal{N}(0, I).$$

Rectified flow defines a linear interpolation between z_0 and z_1:

$$z_t = (1-t)z_0 + tz_1, \qquad t \in [0, 1].$$

A conditional denoising model $f_\theta(z_t, t; x)$ is trained to recover the clean target latent from the intermediate state z_t.[1]

$$\hat{z}_1 = f_\theta(z_t, t; x).$$

The standard x_0-prediction objective is

$$\mathcal{L}_{\text{RF}} = \mathbb{E}\left[\|\hat{z}_1 - z_1\|_2^2\right].$$

This x_0-prediction parameterization implicitly defines a velocity field

$$u_\theta(z_t, t; x) = \frac{f_\theta(z_t, t; x) - z_t}{1-t}.$$

[1]In this work, we use the x_0-prediction parameterization, where the model predicts the clean endpoint rather than the velocity directly. In our latent-space notation, this clean endpoint is denoted by z_1.

At inference we integrate the implied velocity field from $t = 0$ to $t = 1$ with a Heun (second-order Runge-Kutta) solver. The resulting endpoint $\hat{z}_1$ is decoded to produce the final prediction

$$\hat{y} = D_\psi(\hat{z}_1).$$

3 ReasonFlow

3.1 Anchored Trajectory Shaping

The latent-space rectified flow in Section 2 trains the model with a single fixed clean target z_1. For reasoning problems, this creates a direct transport objective: the model is trained to move from noise to the final answer latent without explicit supervision on how the intermediate states should reflect the intermediate reasoning progress. Although the generation process is progressive in flow time, the standard objective does not require it to correspond to the step-by-step structure of a solution.

ReasonFlow modifies this objective by replacing the fixed clean target with a time-dependent target defined by reasoning anchors. Let each training example be given by

$$(x, r, y), \qquad r = (r_1, \ldots, r_K),$$

where x is the input problem, r_k is the k-th reasoning step, and y is the final answer. We introduce a sequence of continuous reasoning anchors

$$z_1^a, \ldots, z_K^a,$$

where z_k^a represents the reasoning state after the prefix $(x, r_1, \ldots, r_k)$.

ReasonFlow aligns the flow time $t \in [0, 1]$ with progress along this reasoning chain. At early times, the model is guided toward early reasoning states; at later times, it is guided toward later reasoning states; and at the end, it reaches a state compatible with the final answer latent. To define the time-dependent clean target, we interpolate between adjacent anchors. For $t \in [0, 1)$, let

$$k = \lfloor tK \rfloor, \qquad s = tK - k.$$

We define

$$z^a(t) = (1 - s)z^a_{k+1} + sz^a_{k+2}.$$

Therefore, at each anchor time $t = k/K$, the clean target coincides with the corresponding reasoning anchor z^a_{k+1}, while between anchor times it moves linearly between adjacent anchors.

This is the simplest curve that (a) starts at z^a_1 at $t = 0$, (b) ends at z_1 at $t = 1$, (c) is continuous in t, and (d) passes through every anchor at the natural time $t = k/K$. Smoothness in t matches the smoothness inductive bias of flow matching; the discrete K-segment structure aligns with the discrete-step structure of CoT. Higher-order curves (cubic Hermite splines, geodesics on the sphere) would add capacity but we leave for future work.

The anchored flow interpolant is

$$z_t = (1 - t)z_0 + tz^a(t), \qquad z_0 \sim \mathcal{N}(0, I).$$

The denoiser uses the same x_0-prediction parameterization as in Section section 2, but the fixed target z_1 is replaced by the time-dependent target $z^a(t)$.

The anchored flow objective is

$$\mathcal{L}_{\text{anchor}} = \mathbb{E}\left[\|f_\theta(z_t, t; x) - z^a(t)\|_2^2\right].$$

Figure 1 illustrates this change. In standard flow matching, all intermediate states are supervised by the same final target z_1, so the vector field points toward the answer latent at all times. In ReasonFlow, the clean target changes with t: the trajectory is pulled toward early reasoning anchors at early denoising times and toward later reasoning anchors at later denoising times. The trajectory is therefore not merely a direct transport path from noise to the answer. Instead, the vector field is shaped by a sequence of reasoning targets, aligning progressive denoising with progressive reasoning.

3.2 Constructing Anchors

The anchored objective above is general: it only requires a sequence of continuous anchors that encode intermediate reasoning states. Different anchor constructors could be used, as long as they map reasoning prefixes into a representation space compatible with the latent flow model. In this work, we instantiate this idea using a causal transformer language model, as illustrated in Figure 2.

Given a training example (x, r, y), we form a token sequence containing the input x, the reasoning steps $r_1, \ldots, r_K$, and the final answer y. A causal language model processes this sequence from left to right. Let $h_i^{(\ell)}$ denote the hidden state at token position i in layer ℓ, and let ℓ_{mid} denote a middle layer of the model. For each reasoning step k, let P_k denote the set of token positions belonging to the sequence $(r_1, \ldots, r_k)$. We construct the k-th anchor by mean-pooling the middle-layer hidden states over this set:

$$\tilde{z}^a_k = \frac{1}{|P_k|} \sum_{i \in P_k} h_i^{(\ell_{\text{mid}})},$$

The anchor is then normalized to the same sphere as the latent flow space:

$$z^a_k = \text{sphere}(\tilde{z}^a_k).$$

This construction gives each anchor a natural causal interpretation. Since the language model is causal, the hidden states used to form z^a_k depend only on the input and the first k reasoning steps. The anchor z^a_k therefore represents the reasoning state available after step k. Mean pooling over the prefix gives a compact representation of the accumulated reasoning state, while using a middle layer avoids tying the anchor too directly to the final token-prediction layer inspired by the information bottleneck theory (Saxe et al., 2018).

The final anchor should also be compatible with the answer latent z_1 from Section 2. Recall that

$$z_1 = \text{sphere}(E_\phi(x, y))$$

is the latent representation from which the decoder reconstructs the final answer. To ensure that the endpoint of the anchor path connects smoothly to this answer space, we add an alignment loss between the last reasoning anchor and the answer latent:

$$\mathcal{L}_{\text{align}} = \lambda \|z^a_K - z_1\|_2^2.$$

This encourages the final reasoning anchor z^a_K to lie near the latent state used for answer decoding.

3.3 Training and Inference

ReasonFlow changes the training target of latent-space rectified flow, but preserves the same inference procedure. Compared with the standard objective in Section 2, the only conceptual change is that the clean target is no longer the fixed answer latent z_1, but the time-dependent anchored target $z^a(t)$ defined in Section 3.1. Thus, chain-of-thought supervision is used only to shape the vector-field targets during training.

At inference time, no reasoning trace is provided and no anchors are constructed. Given only the input x, ReasonFlow starts from noise, integrates the learned velocity field, and decodes the endpoint exactly as in latent-space rectified flow. Therefore, ReasonFlow does not generate chain-of-thought tokens or require intermediate reasoning steps at test time; it keeps the inference process unchanged while learning a vector field whose progressive denoising dynamics are guided by intermediate reasoning states.

4 Experiments

4.1 Tasks and Datasets

We evaluate ReasonFlow on five reasoning tasks spanning arithmetic, formal logic, and chemistry. **GSM8K** (Cobbe et al., 2021) in the formulas-only variant of Deng et al. (2023) that rewrites each chain as a sequence of <<a*b/cd»= calculator equations followed by a numeric answer. **ProntoQA** (Saparov and He, 2023) is a synthetic multi-hop logical-deduction benchmark; we use a hops-4–5 split with 9–11 alternating rule/derived-fact lines per chain. The chemistry cascade tests transfer to harder-to-tokenize domains: **PaRoutes** (Genheden and Bjerrum, 2022) retrosynthesis specifies a target molecule, starting materials, and 2–10 intermediate molecular states as canonical SMILES (Weininger, 1988); **mech-USPTO-31K** (Chen et al., 2024b) provides electron arrow-pushing mechanisms for 31,364 organic reactions drawn from USPTO-50K (Schneider et al., 2016); **MolPuzzle** is a low-data spectroscopy-to-SMILES task (3,936 train) where the chain-of-thought is a substructure-identification trace. Per-task token budgets, dataset sizes, splits, and the pad_to_M pace used at phase-2 are deferred to Section B.

4.2 Architecture and backbones

ReasonFlow uses four Qwen3-0.6B-based modules: (i) the encoder and decoder in autoencoder for z_1, frozen during diffusion training; (ii) the **anchor LM**, finetuned on producing r and then frozen during diffusion training; (iii) a separate **question encoder** that reads $\boldsymbol{x}$ alone and provides per-token KV for cross-attention by the denoiser, trained during diffusion training. The denoiser f_θ is a canonical DiT (Peebles and Xie, 2023) with PixArt-style cross-attention (Chen et al., 2024a) over $H_{\boldsymbol{x}}$. Each module supports full fine-tuning or LoRA r=32 adapters (Hu et al., 2022). All hyperparameters are deferred to Section A.

4.3 Training and inference setup

The salient defaults: the anchor LM finetune is 8,000 steps batch 256 at lr 2×10^{-5}; diffusion training is 200,000 steps batch 64 at lr 5×10^{-5} with $N = 64$ Heun ODE steps. We use the val-EM-best checkpoint (selected by greedy generation on a 200-example val subset every 5,000 steps) to evaluate on test set for main results. Full hyperparameters, optimizer schedules, sphere-norm details, and multi-seed voting protocol are in Section A.

4.4 Baselines

We compare ReasonFlow against two reference points. **Autoregressive LM with CoT**: a standalone Qwen3-0.6B finetuned end-to-end on $[\boldsymbol{x}, \mathrm{r}, \boldsymbol{y}]$ and evaluated by greedy generation conditioned on $\boldsymbol{x}$ alone. This shares the anchor-encoder backbone but uses the LM's full attention/output machinery; it serves as a strong reference upper bound that the flow-matching family is not expected to match. **No-CoT flow matching**: the AE input is $[\boldsymbol{x}, \boldsymbol{y}]$ only; phase-2 trains a DiT to predict the resulting $\boldsymbol{z}_1$ from sphere-normed Gaussian noise. The model never sees CoT at any stage and is the genuine flow-matching baseline against which CoT supervision should be measured.

4.5 Main Results

Table 1 reports greedy single-seed answer EM across the five tasks. The autoregressive LM block is a strong reference upper bound. Within the flow-matching block, ReasonFlow beats no-CoT on every task. The largest gains are on the two tasks where reasoning is most central: +32.0pp on GSM8K and +36.5pp on ProntoQA-harder. ReasonFlow also exceeds no-CoT on the three chemistry tasks (mech +1.5pp, retro +1.5pp, MolPuzzle +5.5pp), with the small-data MolPuzzle task showing the largest relative gain in the chemistry cascade. ReasonFlow recovers between 47.6%

(GSM8K) and 99.6% (mech) of the AR-LM upper bound, with the largest absolute gap remaining on GSM8K, where the AR LM has a strong arithmetic prior that the single-vector anchor cannot fully transmit.

Results of chemistry tasks. On the mechanism task the answer is a near-deterministic atom-rearrangement of the reactant set, which the no-CoT AE answer head can already achieve high accuracy (90.5%); ReasonFlow's anchored trajectory adds a small additional boost (+1.5pp, near the AR-LM ceiling of 92.4%). On retrosynthesis both flow-matching variants and the AR LM are bottlenecked by gold-multiplicity (PaRoutes provides a single canonical route while there are many valid disconnections; see Section I). On MolPuzzle absolute EM is capped by the small training set (3,936 examples), but the relative gain shows that anchored trajectory shaping makes more efficient use of the cot signal than the no-CoT AE.

5 Analysis

5.1 Strict vs. weak no-CoT

The no-CoT baseline used in Table 1 is what we call *strict*: the AE never sees `cot` at any stage. A natural question is whether the gain of ReasonFlow comes from z_1 containing CoT information due to the regularization term. To test this, we compare to *weak* no-CoT, where we use the same z_1 as in ReasonFlow but diffusion training doesn't use the multi-anchor trajectory shaping. Table 2 compares the two on all five tasks.

The result is task-dependent: on GSM8K the weak baseline is dramatically stronger than strict (18.5% vs 1.0%), but still lagging behind the multi-anchor trajectory shaping approach in ReasonFlow; on retrosynthesis and MolPuzzle this even hurt performance; on mech and Pronto the answer is encodable from $[\boldsymbol{x}, \boldsymbol{y}]$ directly, so CoT exposure is roughly inert. ReasonFlow beats both no-CoT variants on every task.

5.2 Reasoning order matters

To check whether the reasoning is being used as semantic structure rather than just additional capacity, we train an additional variant where reasoning sequences are shuffled (derangement within length groups), preserving the distribution of reasoning lengths but breaking the semantic chain. Table 3 reports the impact.

The pattern is consistent with the depth analysis in Section 5.4. On retrosynthesis, where the answer requires the chain to compose, breaking the chain halves the answer EM. On Pronto, where the True/False verdict is largely encoded by the question encoder, shuffling preserves the answer but completely destroys step-level alignment (66.2% $\rightarrow$ 0.0%). The flow learns the proof chain only when the chain is in the right order.

5.3 Number of ODE integration steps

We re-evaluate the GSM8K ReasonFlow checkpoint at $n_{\text{int}} \in \{8, 16, 32, 64, 128\}$ Heun steps (single seed, fixed initial noise). Table 4 reports the answer EM. The number is identical at every n_{int} across a $32\times$ range, indicating that the learned velocity field is very robust and does not require fine-grained discretization. ReasonFlow can be deployed with as few as 8 integration steps without measurable quality loss.

5.4 Reasoning traces "for free" from the trajectory

A trained ReasonFlow consumes the question and produces a final answer; it never explicitly emits any reasoning tokens at inference. A natural question is then whether the intermediate states $\boldsymbol{z}_t$ along the integrated trajectory carry recognizable reasoning content, or whether they are opaque mid-denoising states.

To test this, we trained a suffix decoder to decode at every step boundary $k \in \{0, \ldots, K\}$ to map $(\boldsymbol{x}, \boldsymbol{z}^a_k, k) \rightarrow r[k{:}K_{\text{real}}] + \boldsymbol{y}$. We then feed it the integrated state $\boldsymbol{z}(t = k/K)$ along with conditioning index k. If the trajectory tracks the anchor latents (as the training loss specifies), this should recover step k of the cot. Concatenating the first emitted cot step at each $k = 0, 1, \ldots$ yields an inferred trace. We run this on the GSM8K ReasonFlow ckpt over 50 val examples; per-k exact-match accuracies are deferred to Section G.

Qualitative examples. Table 5 shows three representative inferred traces. The first reasoning step ($k = 0$) is consistently informative (recovered in 44–48% of cases across variants); subsequent decodes either repeat tokens of step 0 or jump straight to a continuation that produces the final answer. Even in the regime where per-step EM is low, the eventual answer-EM at $\boldsymbol{z}(t{=}1)$ is still 28% on this 50-example sample.

Table 1: Main results: greedy single-seed answer EM per task. The autoregressive block reports a standalone Qwen3-0.6B finetuned end-to-end on $[\boldsymbol{x}, r, \boldsymbol{y}]$ and is included as a reference upper bound. Bold marks the best variant within the flow-matching block. EM uses the per-task normalizer with the salvage-prefix extractor of Section A.

Method	**GSM8K**	**Pronto**	**Mech**	**Retro**	**Molpuzzle**
Autoregressive (reference upper bound)					
AR LM with CoT	69.4%	100.0%	92.4%	18.8%	24.6%
Flow matching					
No-CoT	1.0%	53.5%	90.5%	12.0%	14.0%
ReasonFlow	**33.0%**	**90.0%**	**92.0%**	**13.5%**	**19.5%**

Table 2: Strict vs. weak no-CoT per task. *Strict*: AE trained on $[\boldsymbol{x}, \boldsymbol{y}]$ only. *Weak*: we use the same z_1 as in ReasonFlow but diffusion training doesn't use the multi-anchor trajectory shaping. The signed $\Delta =$ Weak$-$Strict tells us whether cot exposure helps (positive) or hurts (negative) $\boldsymbol{z}_1$ for the no-CoT recipe.

Task	**Strict**	**Weak**	Δ
GSM8K	1.0%	18.5%	+27.5pp
Retrosynthesis	13.5%	2.5%	−11.0pp
Mech-USPTO-31K	92.0%	91.5%	− 0.5pp
Molpuzzle	19.5%	2.0%	−17.5pp
ProntoQA	53.5%	53.0%	− 0.5pp

Table 3: Effect of shuffling the reasoning chain at training time. Shuffled-order training preserves the distribution of cot lengths but destroys the semantic chain.

Task / metric	**Gold**	**Shuffled**	**Drop**
Retrosynthesis	18.8%	10.2%	−8.6pp
Pronto answer	100.0%	95.0%	−5.0pp
Pronto step	66.2%	0.0%	−66.2pp

Interpretation. This is, to our knowledge, the first concrete demonstration that a continuous flow trained with anchored reasoning supervision carries decodable reasoning semantics in its early-trajectory states. The current recipe extracts the first reasoning step "for free" but does not yet produce a complete step-by-step trace; tightening the trajectory's tracking of intermediate $\boldsymbol{z}_k^a$ (e.g. via per-segment auxiliary losses or stricter anchor-passage constraints) is a clear path forward. We view the k=0 result as evidence that the framework's intermediate states are not opaque denoising checkpoints but partially recoverable reasoning waypoints—preserving the interpretability advantage of discrete reasoning while remaining in the continuous-time flow framework.

Table 4: Answer EM on GSM8K (seed= 42) at varying ODE-integration step counts. The bigdata mean-pool checkpoint produces the same answer at every n_{int}, demonstrating that the learned velocity field is nearly straight. ReasonFlow can be deployed with 16× less inference compute than the default.

n_{int}	8	16	32	64	128
EM	33.0%	33.0%	33.0%	33.0%	33.0%

5.5 Are correct answers backed by correct reasoning?

A natural follow-up is whether the answer the model produces is *caused* by the reasoning it traces, or whether the two are decoupled. We compute the joint distribution of (cot-step correctness, answer correctness) over the same 50 examples on GSM8K, for each anchored variant. Table 6 reports two granularities: full-cot match (every gold step recovered in order) and first-step match (only the k=0 predicted step matches gold).

Full-CoT correctness is essentially zero (cell A $= 0/50$ across all three variants). The trajectory does not faithfully reproduce gold cot end-to-end. Out of 50 examples, exactly one had a fully correct cot trace, and that one example had the wrong answer (cell B$= 1$).

Even incorrect reasoning might lead to correct answers ("right answer for wrong reason," cell C). For mean pool and Joint, $14/14$ correct answers (28% of all examples) had at least one wrong intermediate step. The flow's $\boldsymbol{z}(t{=}1)$ encodes enough of the answer to be decodable correctly even when the trajectory's intermediate states are incorrect. This is expected: as shown in Figure 1, even the ideal trajectory does not necessarily pass through every anchor point; the anchor points only bias the trajectory to stay close to them.

First-step correctness predicts answer correctness more strongly than full-CoT. On mean pool,

Table 5: Three inferred reasoning traces from ReasonFlow (mean pool, GSM8K). At $k{=}0$, the integrated state $\boldsymbol{z}(t{=}0)$ decodes to the correct first reasoning step in all three. For $k{\geq}1$ the trace becomes degenerate (continuations of step 0), yet the final $\boldsymbol{z}(t{=}1)$ still produces the correct answer in two of three cases.

Q: *Hannah has three dogs. The first dog eats 1.5 cups a day. The second eats twice as much, the third eats 2.5 more than the second. How many cups for all three?*
Gold cot: `<<1.5*2=3>> <<3+2.5=5.5>> <<1.5+3+5.5=10>>` **Gold ans:** 10
Trace: $k{=}0$:`<<1.5*2=3>>` $k{=}1$:`1.5*2=3>>` $k{=}5$:`<<3+3+5.5=11.5>>` **Pred ans: 10** ✓

Q: *Travis wants to fly to Australia. Tickets cost $2000. Student discount of 30%. How much pays?*
Gold cot: `<<30/100*2000=600>> <<2000-600=1400>>` **Gold ans:** 1400
Trace: $k{=}0$:`<<2000*30/100=600>>` $k{=}1{-}5$: continuations $k{=}6$:`<<2000-600=1400>>` **Pred ans: 1400** ✓

Q: *John cuts grass to 2 inches. It grows 0.5/month. He cuts back at 4 inches, costs $100 each. How much pays per year?*
Gold cot: `<<4-2=2>> <<2/.5=4>> <<12/4=3>> <<100*3=300>>` **Gold ans:** 300
Trace: $k{=}0$:`<<4-2=2>>` $k{=}2$:`*12=12>>` $k{=}4$:`<<12*100=1200>>` **Pred ans:** 100 (×)

Table 6: Joint correctness on GSM8K. *full cot*: predicted step matches every gold step in order; *first step*: $k{=}0$ predicted step matches gold $k{=}0$. A: cot+ans both right (ideal). B: cot right, ans wrong. C: cot wrong, ans right (right answer for wrong reason). D: both wrong.

	full-cot × ans				first-step × ans			
Variant	A	B	C	D	A	B	C	D
mean	0	1	14	35	4	18	10	18
last_token	0	1	11	38	4	19	7	20
Joint	0	1	14	35	7	17	7	19

$4/22 = 18\%$ of examples with a correct $k{=}0$ trace get the right answer (cell $\mathrm{A}_{\text{first}}/(\mathrm{A{+}B})_{\text{first}}$); $10/28 = 36\%$ of examples with a wrong $k{=}0$ trace get the answer right anyway. The first-step extraction is a useful interpretability surface but it is not a faithful mediator of the model's prediction.

This has direct interpretability implications: the trajectory analysis of Section 5.4 should be read as a *partial* explanation of how the flow arrives at its answer, not a faithful reasoning trace.

6 Conclusion

We have shown that flow matching can be guided to reason step by step. The key mechanism is anchored trajectory shaping: instead of training the model toward a single fixed clean target, ReasonFlow replaces the target with a piecewise-linear path through CoT-induced reasoning anchors. These anchors align the flow-time axis with the structure of the reasoning process, so intermediate denoising states are pulled toward corresponding reasoning waypoints before reaching the final answer latent. With anchors constructed from a finetuned Qwen3-0.6B model, ReasonFlow improves over standard no-CoT flow matching on all five reasoning tasks we evaluate.

Limitations

Gap to autoregressive reasoning. Although ReasonFlow improves over no-CoT flow-matching baselines, it still lags behind autoregressive language models. This gap reflects a broader challenge for fixed-dimensional continuous generative models: reasoning may require variable-length memory, while flow matching compresses the reasoning state into continuous latent vectors. Nevertheless, ReasonFlow is a step toward closing this gap by introducing intermediate reasoning supervision into the flow trajectory.

Anchor and trajectory design. Our current trajectory design is intentionally simple: we use a piecewise-linear path through reasoning anchors to align flow time with reasoning step index. Future work could explore higher-order curves, geodesic interpolation on the sphere, learned interpolants, or adaptive pacing over reasoning steps. Similarly, our anchors are constructed from mean-pooled hidden states of a finetuned causal language model. This is only one convenient instantiation, and specialized anchor encoders or jointly learned anchor representations may provide stronger supervision for continuous-time reasoning.

Trajectory interpretability. Anchored trajectory shaping gives intermediate flow states more semantic structure, but it does not guarantee that the generated trajectory is a faithful explanation of the final answer. The anchors guide the vector field during training, but the model is not constrained to produce an explicitly verifiable reasoning chain at inference time, as reflected in our trajectory analyses. Developing stronger constraints and diagnostics for faithful continuous reasoning remains an important direction.

Ethical Considerations

This work proposes a general method for improving reasoning in flow-matching models and is not designed for a specific high-risk deployment setting. As with other methods that improve reasoning capability, the approach could eventually contribute to both beneficial and harmful applications depending on deployment context. In addition, although ReasonFlow uses reasoning traces to shape intermediate trajectories, these trajectories should not be treated as guaranteed faithful explanations of the final answer without task-specific validation.

Data content. All datasets we use are public research benchmarks that do not contain personally identifying information or offensive content. GSM8K consists of arithmetic word problems; ProntoQA is synthetically generated with fictional ontologies (made-up entities and properties) and therefore contains no information about real people; PaRoutes and mech-USPTO-31K contain canonical SMILES strings for chemical reactions; MolPuzzle contains NMR / IR / mass-spectrum inputs paired with target SMILES. Because none of these data sources involve human subjects or contain text describing real individuals, no anonymization or content-filtering step was required.

Use of AI assistants. During the preparation of this work, the authors used AI coding assistants for code drafting help and AI writing assistants for manuscript polishing. All scientific claims, experimental designs, hyperparameter choices, results, tables, figures, and final manuscript text were reviewed, verified, and approved by the authors. No AI-generated material was used as a citable source. AI assistants did not contribute to research ideation, experimental design decisions, or interpretation of the results.

References

Michael S. Albergo, Nicholas M. Boffi, and Eric Vanden-Eijnden. 2023. Stochastic interpolants: A unifying framework for flows and diffusions. *arXiv preprint arXiv:2303.08797*.

John Bradshaw, Matt J. Kusner, Brooks Paige, Marwin H. S. Segler, and José Miguel Hernández-Lobato. 2019. A generative model for electron paths. In *International Conference on Learning Representations*.

Andrew Campbell, Joe Benton, Valentin De Bortoli, Tom Rainforth, George Deligiannidis, and Arnaud Doucet. 2022. A continuous time framework for discrete denoising models. In *Advances in Neural Information Processing Systems*.

Binghong Chen, Chengtao Li, Hanjun Dai, and Le Song. 2020. Retro*: Learning retrosynthetic planning with neural guided A* search. In *International Conference on Machine Learning*.

Junsong Chen, Jincheng Yu, Chongjian Ge, Lewei Yao, Enze Xie, Yue Wu, Zhongdao Wang, James Kwok, Ping Luo, Huchuan Lu, and Zhenguo Li. 2024a. PixArt-α: Fast training of diffusion transformer for photorealistic text-to-image synthesis. In *International Conference on Learning Representations*.

Shuan Chen, Ramil Babazade, Taewan Kim, Sunkyu Han, and Yousung Jung. 2024b. A large-scale reaction dataset of mechanistic pathways of organic reactions. *Scientific Data*, 11:863.

Shuan Chen, Kye Sung Park, Taewan Kim, Sunkyu Han, and Yousung Jung. 2025. Predicting chemical reaction outcomes based on electron movements using machine learning. *arXiv preprint arXiv:2503.10197*.

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. 2021. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*.

Antonia Creswell, Murray Shanahan, and Irina Higgins. 2023. Selection-inference: Exploiting large language models for interpretable logical reasoning. In *International Conference on Learning Representations*.

Yuntian Deng, Kiran Prasad, Roland Fernandez, Paul Smolensky, Vishrav Chaudhary, and Stuart Shieber. 2023. Implicit chain of thought reasoning via knowledge distillation. *arXiv preprint arXiv:2311.01460*.

Samuel Genheden and Esben Jannik Bjerrum. 2022. PaRoutes: Towards a framework for benchmarking retrosynthesis route predictions. *Digital Discovery*, 1:527–539.

Shibo Hao, Sainbayar Sukhbaatar, DiJia Su, Xian Li, Zhiting Hu, Jason Weston, and Yuandong Tian. 2024. Training large language models to reason in a continuous latent space. *arXiv preprint arXiv:2412.06769*.

Jonathan Ho, Ajay Jain, and Pieter Abbeel. 2020. Denoising diffusion probabilistic models. In *Advances in Neural Information Processing Systems*.

Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2022. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*.

Joonyoung F. Joung, Mun Hong Fong, Nicholas Casetti, Jordan P. Liles, Ne S. Dassanayake, and Connor W. Coley. 2025. Electron flow matching for generative reaction mechanism prediction. *Nature*, 645:115–123.

Tero Karras, Miika Aittala, Timo Aila, and Samuli Laine. 2022. Elucidating the design space of diffusion-based generative models. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. 2023. Let's verify step by step. *arXiv preprint arXiv:2305.20050*.

Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. 2023. Flow matching for generative modeling. In *International Conference on Learning Representations*.

Xingchao Liu, Chengyue Gong, and Qiang Liu. 2023. Flow straight and fast: Learning to generate and transfer data with rectified flow. In *International Conference on Learning Representations*.

Aaron Lou, Chenlin Meng, and Stefano Ermon. 2024. Discrete diffusion modeling by estimating the ratios of the data distribution. In *International Conference on Machine Learning*.

Théo A. Neukomm, Zlatko Jončev, and Philippe Schwaller. 2025. Teaching language models mechanistic explainability through MechSMILES. *arXiv preprint arXiv:2512.05722*.

Maxwell Nye, Anders Johan Andreassen, Guy Gur-Ari, Henryk Michalewski, Jacob Austin, David Bieber, David Dohan, Aitor Lewkowycz, Maarten Bosma, David Luan, Charles Sutton, and Augustus Odena. 2021. Show your work: Scratchpads for intermediate computation with language models. *arXiv preprint arXiv:2112.00114*.

William Peebles and Saining Xie. 2023. Scalable diffusion models with transformers. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*.

Qwen Team. 2025. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*.

Abulhair Saparov and He He. 2023. Language models are greedy reasoners: A systematic formal analysis of chain-of-thought. In *International Conference on Learning Representations*.

Andrew Michael Saxe, Yamini Bansal, Joel Dapello, Madhu Advani, Artemy Kolchinsky, Brendan Daniel Tracey, and David Daniel Cox. 2018. On the information bottleneck theory of deep learning. In *International Conference on Learning Representations*.

Nadine Schneider, Nikolaus Stiefl, and Gregory A. Landrum. 2016. What's what: The (nearly) definitive guide to reaction role assignment. *Journal of Chemical Information and Modeling*, 56:2336–2346.

Philippe Schwaller, Riccardo Petraglia, Valerio Zullo, Vishnu H. Nair, Rico Andreas Haeuselmann, Riccardo Pisoni, Costas Bekas, Anna Iuliano, and Teodoro Laino. 2020. Predicting retrosynthetic pathways using transformer-based models and a hyper-graph exploration strategy. *Chemical Science*, 11:3316–3325.

Marwin H. S. Segler, Mike Preuss, and Mark P. Waller. 2018. Planning chemical syntheses with deep neural networks and symbolic AI. *Nature*, 555:604–610.

Chence Shi, Minkai Xu, Hongyu Guo, Ming Zhang, and Jian Tang. 2020. A graph to graphs framework for retrosynthesis prediction. In *International Conference on Machine Learning*.

Yang Song, Jascha Sohl-Dickstein, Diederik P. Kingma, Abhishek Kumar, Stefano Ermon, and Ben Poole. 2021. Score-based generative modeling through stochastic differential equations. In *International Conference on Learning Representations*.

Oyvind Tafjord, Bhavana Dalvi Mishra, and Peter Clark. 2021. Proofwriter: Generating implications, proofs, and abductive statements over natural language. In *Findings of the Association for Computational Linguistics: ACL-IJCNLP 2021*, pages 3621–3634.

Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc V. Le, Ed H. Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. 2023. Self-consistency improves chain of thought reasoning in language models. In *International Conference on Learning Representations*.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed H. Chi, Quoc V. Le, and Denny Zhou. 2022. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems*.

David Weininger. 1988. SMILES, a chemical language and information system. 1. introduction to methodology and encoding rules. *Journal of Chemical Information and Computer Sciences*, 28(1):31–36.

Jiacheng Ye, Shansan Gong, Liheng Chen, Lin Zheng, Jiahui Gao, Han Shi, Chuan Wu, Xin Jiang, Zhenguo Li, Wei Bi, and Lingpeng Kong. 2024. Diffusion of thought: Chain-of-thought reasoning in diffusion language models. In *Advances in Neural Information Processing Systems*.

Eric Zelikman, Georges Harik, Yijia Shao, Varuna Jayasiri, Nick Haber, and Noah D. Goodman. 2024. Quiet-STaR: Language models can teach themselves to think before speaking. *arXiv preprint arXiv:2403.09629*.

Eric Zelikman, Yuhuai Wu, Jesse Mu, and Noah D. Goodman. 2022. STaR: Bootstrapping reasoning with reasoning. In *Advances in Neural Information Processing Systems*.

A Implementation Details

Sphere normalization. We apply $\text{sphere}(\boldsymbol{z}) = \sqrt{d} \cdot \boldsymbol{z}/\|\boldsymbol{z}\|$ at every projection that produces a latent: at the AE output ($\boldsymbol{z}_1$), at sampled noise ($\boldsymbol{z}_0 = \text{sphere}(\boldsymbol{\epsilon})$), and at the denoiser output ($\hat{\boldsymbol{z}}_1$).

Backbone configurations. All Qwen3-0.6B trunks (f_T, f_Q, f_C, the anchor LM, and the Joint-variant encoder/decoder) support either full fine-tuning or LoRA r=32 adapters over the seven projection modules $\{q, k, v, o, \text{gate}, \text{up}, \text{down}\}$ of every transformer block, with $\alpha = 32$, dropout 0.0. We use full fine-tuning for the headline GSM8K runs and LoRA r=32 for the chemistry / logic transfer cascade where memory is more constrained. `gradient_checkpointing` is enabled on all trunks.

Stage-1 AE training hyperparameters. AdamW lr $= 2 \times 10^{-5}$, weight decay 0.02, cosine annealing with 1,000-step linear warmup. Per-GPU batch size 256 on a single H200. 8,000 steps total, $\sim$ 12 GPU-hours per cell. Latent dim $d = 256$. Sphere-norm enforced. AE noise: $\sigma \sim \mathcal{U}(0, 0.3)$ sampled per example, applied as $\boldsymbol{z}_1 \to \text{sphere}(\boldsymbol{z}_1 + \sigma\,\boldsymbol{\epsilon})$ before f_A. Pool mode: `last_token` by default; `mean` also tested in ablations.

Stage-2 phase-2 training hyperparameters. AdamW lr $= 5 \times 10^{-5}$, weight decay 0.02, cosine annealing with 2,000-step linear warmup. Per-GPU batch size 64, 200,000 steps for headline runs. Float32 precision throughout (the velocity-field magnitudes near $t \to 1$ require it). Gradient clipping at norm 1.0. Question encoder f_Q is initialized fresh from Qwen3-0.6B-Base (NOT from the AE-trained f_T) and trained from scratch alongside the denoiser.

DiT denoiser. We use the canonical DiT (Peebles and Xie, 2023) architecture with PixArt-style ungated cross-attention (Chen et al., 2024a). Sweep configurations: DiT-S (hidden=384, depth=12, heads=6, ff=4×); DiT-B (hidden=768, depth=12, heads=12, ff=4×); DiT-L (hidden=1024, depth=24, heads=16, ff=4×); DiT-XL (hidden=1152, depth=28, heads=16, ff=4×). Latent-token count $n_{\text{lat}} \in \{1, 16\}$.

Inference hyperparameters. Heun (second-order Runge-Kutta) integrator with $N = 64$ steps from $t = 0$ to $t = 1$. The implied velocity field is $\boldsymbol{u}_\theta(\boldsymbol{z}_t, t) = (\hat{\boldsymbol{z}}_1 - \boldsymbol{z}_t)/(1 - t)$ where $\hat{\boldsymbol{z}}_1$ is the sphere-normed denoiser output. We clamp the denominator $(1-t)$ to $\geq 10^{-3}$ for numerical stability at $t \to 1$.

Multi-seed voting protocol. We run inference $K = 19$ times with $\boldsymbol{\epsilon}$-seeds $\{0, 1, \ldots, 18\}$, normalize each decoded answer string with the per-task normalizer (numeric / smiles / set_smiles / classify), and aggregate by majority vote.

Variant-A pad_to_M schedule. For each task we set M to roughly the p90 of the training cot length under that task's tokenizer (GSM8K: $M = 48$; retro: $M = 208$; mech: $M = 176$; MolPuzzle: $M = 328$; ProntoQA: $M = 104$; see Table 7). At training, $k = \lfloor t \cdot M \rfloor$ for sampled $t \sim \mathcal{U}(0, 1)$; loss positions with $k \geq L_{\text{cot}}$ are masked.

Per-task M for the anchored phase-2. The anchored variant of ReasonFlow extracts $K{+}1 = M{+}1$ anchors per example and trains the DiT against the time-varying $\boldsymbol{z}^a(t)$ that interpolates them. Empirically the DiT can track piecewise-linear paths through tens of waypoints comfortably; hundreds is harder. We therefore use the dataset-level M for tasks with moderate cot length (GSM8K: $M = 48$; ProntoQA: $M = 104$; mech: $M = 176$; retro: $M = 208$) but reduce MolPuzzle's M to 64 for the anchored phase-2 only (the anchor LM was finetuned with the full cot $M = 328$; the smaller phase-2 M just samples fewer waypoints). The dataset-level M remains 328 for the no-CoT baselines.

SMILES-aware answer extractor. For SMILES-output tasks (mech, retro, MolPuzzle), the SuffixDecoder emits `cot[k:K] #### ans` and at $k{=}K_{\text{total}}$ would ideally emit only `ans`. In practice the decoder regularly regurgitates a few extra cot-style tokens (e.g. `1), ||| ([301, 1], 1))`) after the answer, corrupting the SMILES and causing canonicalization to fail. We therefore use a salvage extractor: the predicted answer is the longest right-trimmed prefix (up to 32 trailing characters removed, with `() , . \s` also stripped) that RDKit `MolFromSmiles` parses successfully, followed by canonicalization. This is what closes the prior measurement gap on mech (56% $\to$ 92.0%) and lifts retro and MolPuzzle by 1–3pp each. Numeric (gsm8k) and classify (prontoqa) extractors do not use this path. The implementation is the `_canonicalize_one` routine in our answer-normalizer module.

Strict no-CoT AE training. The strict no-CoT baseline (Section 5.1) requires a separate AE per task whose input is $[\boldsymbol{x}, \text{sep2}, \boldsymbol{y}]$ with no reasoning. We expose this via a `-no_cot_ae true` flag in the AE training script that switches the dataset's `tgt_text` construction:

```
if no_cot_ae:
    tgt_text = q + " #### " + ans
else:
    tgt_text = q + " => " + cot
                 + " #### " + ans
```

All other AE hyperparameters are unchanged from the full-triple AE. The phase-2 cell that consumes a strict-AE checkpoint also passes `-no_cot_ae true` so the dataset emits the matching `tgt_ids` format.

B Per-task data details

Table 7 summarizes per-task token budgets, dataset sizes, the pad_to_M supervision pace used at phase-2, and the answer-string normalizer used for exact-match scoring (`numeric`: numeric-string normalization; `smiles`: RDKit canonical SMILES; `set_smiles`: set equality after RDKit canonicalization; `classify`: lower-cased label string).

PaRoutes retrosynthesis. We use the PaRoutes benchmark (Genheden and Bjerrum, 2022) with 150,238 training routes whose targets are disjoint from the n_1/n_5 stock-level evaluation benchmarks. Each route specifies a target molecule, starting materials, and intermediate molecular states (reasoning steps) from retrosynthetic disconnections; routes have 2–10 steps. Molecules are represented as canonical SMILES (Weininger, 1988). We evaluate on the field-standard PaRoutes n_1 and n_5 stock-level benchmarks (1,000 routes each).

Reaction mechanism prediction (mech-USPTO-31K). We use mech-USPTO-31K (Chen et al., 2024b), a Data Descriptor that provides arrow-pushing mechanism annotations for 31,364 organic reactions drawn from USPTO-50K (Schneider et al., 2016) and validated by a panel of ten organic chemists. Each reaction specifies reactants, products, and the electron arrow-pushing mechanism as an ordered sequence of (source, sink) tuples where source is a lone pair (atom index) or a bond (atom-pair), and sink is an atom or a bond. We evaluate on a 1,000-reaction subset of the official held-out split.

Molpuzzle. Spectroscopy-to-structure prediction with NMR / IR / mass-spec inputs and a single SMILES output (3,936 train / 207 val / 130 test problems). Reasoning steps are an interpretable chain of substructure identifications and atom-by-atom assembly. We augment the original MolPuzzle dataset by (1) extracting NMR data from NMR-ShiftDB and (2) getting functional groups information using `RDKit`.

ProntoQA. We use ProntoQA (Saparov and He, 2023), a synthetic logical-reasoning benchmark. Each problem specifies a small ontology of natural-language facts and rules, a query, and a True/False answer. The reasoning steps are the ground-truth derivation chain. We generate 4,000 train / 500 val / 500 test problems using the official ProntoQA generator with fictional-ontology mode and proof depths of 4–5 hops, yielding chains of 9 or 11 alternating rule/derived-fact lines (mean $\approx$9.6). We use this hops-4–5 split because the easier hops-1–3 setting saturates all methods at $\geq$0.986 True/False accuracy and provides no discriminative signal.

C GSM8K formulas-only variant

The headline arithmetic results in this paper use the formulas-only variant of GSM8K (Cobbe et al., 2021) introduced by Deng et al. (2023), where each problem's reasoning chain is rewritten as a sequence of `<<a*b/cd»=` calculator equations followed by a numeric answer. The dataset has 385,620 training problems and 494 each in val and test (the same val/test split shared with the canonical GSM8K). For the bigdata headline (Section D) we use a 1.385M-example synthetic-augmented version of the same training distribution. Token budgets are tight (max_q= 192, cot_max= 80, ans_max= 24 under the Qwen3 tokenizer), and the pace $M = 48$ covers the p90 of cot lengths.

D Standalone GSM8K results

For completeness we report the four-configuration GSM8K-only comparison that the main-text Table 1 subsumes. The four configurations on GSM8K are: **No-CoT**: AE input $[\boldsymbol{x}, \boldsymbol{y}]$ only, never sees cot. **Weak no-CoT**: AE input is the full triple, phase-2 has $w_{\text{tok}} = 0$ (Section 5.1). **ReasonFlow (mean pool)**: anchored phase-2 with mean-pool anchors on the 1.385M combined train set. **AR LM upper bound**: standalone Qwen3-0.6B AR LM finetuned on $[\boldsymbol{x}, \text{cot}, \boldsymbol{y}]$, greedy single-seed.

Table 7: Per-task configuration. M is the pad_to_M supervision pace at phase-2.

Task	Train	Val	Test	max_q	cot_max	ans_max	M	Normalizer
GSM8K	385 620	494	494	192	80	24	48	numeric
PaRoutes retro	150 238	2 411	298	192	448	96	208	set_smiles
Mech-USPTO-31K	25 065	3 108	3 191	368	336	328	176	smiles
Molpuzzle	3 936	207	130	432	352	32	328	smiles
ProntoQA	4 000	500	500	224	128	24	104	classify

Table 8: Standalone GSM8K results. Single-seed greedy answer EM ($n = 200$ for ours, $n = 494$ for weak no-CoT).

Configuration	EM
No-CoT	1.0%
Weak no-CoT (vote@13)	22.27%
ReasonFlow (mean pool)	**33.0%**
AR LM with CoT	69.0%

E Anchored design space

Table 9 consolidates EM across the major design knobs we swept on each task. We report the best variant within each ablation family (selected by EM among 4–12 cells per family); the underlying per-cell numbers are in the released artifacts. We additionally report a `Joint` variant in which the anchor encoder and the suffix decoder are trained jointly, in contrast to our default frozen-LM encoder. Three patterns hold across tasks. (1) The optimal pool mode is task-dependent: `mean` for GSM8K and retro, `mean_cot_only` for mech / MolPuzzle / pronto. (2) DiT capacity is task-saturating: smaller DiT (`ditS` / `ditB`) matches `ditXL` within $\pm$1pp on every task, indicating the bottleneck is anchor information not denoiser capacity. (3) Anchor-dimension projection (truncate or PCA-style SVD down to $D \in \{64, 128, 256, 384, 512\}$ from the native $D = 1024$) is essentially free: every projected dim recovers the native baseline within sampling noise. The trajectory shape ablations on GSM8K (replacing $\boldsymbol{z}_0^a$ with noise; appending $\boldsymbol{z}_K^a \to \boldsymbol{z}_1$ where $\boldsymbol{z}_1$ is the anchor LM's encoding of $[\boldsymbol{x}, \mathrm{cot}, \boldsymbol{y}]$ or an answer-only AE encoding) all land within 0.27–0.31 EM, vs the $\boldsymbol{z}_0^a \to \boldsymbol{z}_K^a$ baseline at 0.32. The takeaway is that no single knob within the V1 frozen-LM family changes the picture; the bottleneck is the anchor representation itself, not pool mode / DiT size / projection.

F Anchor non-degeneracy diagnostic

A natural concern with anchored trajectory shaping is that the anchor sequence might collapse: if all $\boldsymbol{z}_k^a$ lie on top of each other, the time-varying clean target reduces to a constant and the recipe degenerates. We measure four geometric quantities of the anchor sequence over 200 GSM8K examples (M=48, D=1024, sphere-normed): mean cosine similarity between adjacent anchors $\boldsymbol{z}_k^a$ and $\boldsymbol{z}_{k+1}^a$, relative adjacent-step magnitude $\|\boldsymbol{z}_{k+1}^a - \boldsymbol{z}_k^a\|/\sqrt{D}$, trajectory diameter $\|\boldsymbol{z}_K^a - \boldsymbol{z}_0^a\|/\sqrt{D}$, and the fraction of segments with $\|\boldsymbol{z}_{k+1}^a - \boldsymbol{z}_k^a\| < 0.01\sqrt{D}$. The trajectory has adjacent-anchor cosine 0.725 (63° apart on the sphere), relative adjacent-step magnitude 0.586, trajectory diameter $0.85\sqrt{D}$, and only 36.9% of segments fall under the $0.01\sqrt{D}$ near-zero threshold (most of those are the replicated padding segments at $k > L_{\mathrm{cot}}$ in the pad_to_M schedule). The trajectory traverses a substantial fraction of the sphere and gives the suffix decoder a meaningful per-step signal, which is what lets the per-step CoT decoding analysis of Section 5.4 extract a recognizable first reasoning step at k=0.

G Per-step suffix-decoder accuracy

Table 10 reports per-k exact-match accuracy of the suffix-decoder-extracted reasoning trace from ReasonFlow's integrated trajectory on GSM8K (50 examples). At k=0 all three variants recover the gold first reasoning step in 44–48% of cases, well above the marginal probability of any specific $\langle\langle\cdot\rangle\rangle$ expression. For $k\geq$1 exact step EM drops to 0%; with a more lenient match on the inner expression, the Joint variant still recovers 15.8% of the second step. The integrated trajectory hits $\boldsymbol{z}_0^a$ accurately but then collapses toward the final answer state $\boldsymbol{z}_K^a$, rather than carefully threading every intermediate anchor.

Transfer-task reasoning trace extraction. We ran the same trace extraction on the best anchored ckpt per transfer task (50 examples each, n_{int}=K Heun steps, single seed). The arithmetic step-

Table 9: Cross-task ablation grid: EM of the best variant within each design-knob family. "–" means we did not run that family for that task. Bold = task winner across all our experiments.

Knob (best within family)	GSM8K	retro	mech	MolPuzzle	pronto
Pool: mean	0.320	**0.135**	0.895	0.055	0.495
Pool: last_token	0.300	0.070	0.885	**0.195**	0.875
Pool: mean_cot_only	0.240	0.100	**0.920**	0.130	**0.900**
DiT size ∈ {S,B,L} (best)	–	0.090	0.885	0.090	–
M sweep (best M)	–	0.095	0.895	0.195	–
Projection truncate (best D)	–	0.085	0.885	0.095	0.870
Projection SVD (best D)	–	0.090	0.885	0.095	0.885
Recipe sweep (best lr/batch)	–	0.085	0.885	0.090	–
Joint encoder	0.280	0.060	0.895	0.080	0.760
$\boldsymbol{z}_1$ = ans-AE	0.285	0.070	0.885	0.090	0.875
$\boldsymbol{z}_0$ = noise (trajectory shape)	0.300	–	–	–	–
$\boldsymbol{z}_1$ = endofans (trajectory shape)	0.295	–	–	–	–
Bigdata 1.385M (mean pool)	**0.330**	–	–	–	–

Table 10: Per-k exact-match accuracy of the suffix-decoder-extracted reasoning trace from ReasonFlow's integrated trajectory on GSM8K (50 examples).

k	mean	last_token	Joint
k=0	44.0%	46.0%	**48.0%**
k=1 (strict)	0.0%	0.0%	0.0%
k=1 (lenient)*	8.3%	5.7%	15.8%
$k\geq 2$	0.0%	0.0%	0.0%

* Lenient match: equality of the inner expression text (ignoring the $\langle\langle\cdot\rangle\rangle$ wrapper) to allow for the decoder having emitted a partial bracket.

extractor used in Table 10 is GSM8K-specific. As a sanity check, we report the final-answer EM under this trace-extraction integration regime: retro 0.18, mech 0.58, MolPuzzle 0.12, ProntoQA 1.00 (the trace-extraction integration grid uses $n_{\text{int}} = K$ rather than the deployment default $n_{\text{int}} = 64$, which explains the gap from Table 1 on tasks with very large K).

H GSM8K qualitative trace examples

Table 11 reproduces four traces from the mean-pool ckpt on GSM8K (single-seed, $n_{\text{int}} = K = 48$, the same setup as Tables 6 and 10). The four examples are drawn from each cell of the joint-correctness table: ideal (A; absent in our 50-example sample so we instead show the only B example), right answer for wrong reason (C), and catastrophic (D).

I Retrosynthesis qualitative examples

PaRoutes provides a single canonical disconnection per molecule, but most products admit multiple valid retrosynthetic pathways. The strict EM metric counts an alternative valid route as wrong. Table 12 shows three predictions from ReasonFlow on retrosynthesis that strict EM marks wrong but that produce valid alternative starting-material sets.

J Mech qualitative examples (the copy-task structure)

Table 13 illustrates why mech-USPTO-31K is essentially a copy-with-rearrangement task: the atom-mapping numbers $:1, :2, \ldots$ are preserved across reactant→product, so the model only needs to learn the bond-reconnection rule. Both strict no-CoT and ReasonFlow emit the gold product byte-identically in $\sim$90% of cases.

K Datasets, Licenses, and Intended Use

We document below the license and intended use of every existing artifact we rely on, and the intended use of the artifacts we release.

GSM8K (formulas-only variant). We use the formulas-only rewrite of GSM8K (Cobbe et al., 2021) introduced by Deng et al. (2023). GSM8K is released under the MIT license, and the formulas-only derivation has been distributed under the same terms by its authors. Our use as a research benchmark for reasoning models is consistent with the dataset's intended use.

ProntoQA. ProntoQA (Saparov and He, 2023) provides an official generator under an open-source license (Apache-2.0). We use the generator in its intended fictional-ontology mode at hops 4–5. The generated examples contain no information about real people or real-world entities.

Table 11: GSM8K trace examples. **B**: cot fully correct, answer wrong (only example in the 50-row sample). **C**: cot wrong, answer right (right answer for wrong reason). **D**: both wrong. The trace shows the first three predicted cot steps; later steps continue the same pattern.

Cell B (cot correct, answer wrong) – example 15.
Q: *Elsa and her sister watch an opera show. Last year, opera seating cost $85 per ticket. This year it costs $102. Percent increase?*
Gold cot: `<<102-85=17>>` **Gold ans:** 20
Trace: k=0:`<<102-85=17>>` k=1:`102-85=17>>` k=2:`02-85=17>>` **Pred ans:** 15.45 (×)
Reading: the trajectory tracks the first cot step verbatim, then degenerates by character-level shifting; the final answer is computed as $17/110 \cdot 100 \approx 15.45$ rather than the gold $17/85 \cdot 100 = 20$. The cot trace was correct but the readout disagrees.

Cell C (cot wrong, answer right) – example 1.
Q: *Hannah has three dogs. First eats 1.5 cups, second twice as much, third 2.5 more than second. Total per day?*
Gold cot: `<<1.5*2=3>>` `<<3+2.5=5.5>>` `<<1.5+3+5.5=10>>` **Gold ans:** 10
Trace: k=0:`<<1.5*2=3>>` k=1:`1.5*2=3>>` k=5:`=3>>` `<<3+3+5.5=11.5>>` **Pred ans: 10** (✓)
Reading: only the first cot step is reproduced cleanly; later k degenerate. The final answer is correct, indicating that $\boldsymbol{z}(t{=}1)$ encodes the answer despite the trajectory not faithfully traversing intermediate $\boldsymbol{z}_k^a$.

Cell C – example 4.
Q: *Tom bought games for $200, tripled in value, sold 40%. How much did he sell for?*
Gold cot: `<<200*3=600>>` `<<600*.4=240>>` **Gold ans:** 240
Trace: k=0:`<<200*3=600>>` k=1:`200*3=600>>` k=4:`<<600*40/100=240>>` **Pred ans: 240** (✓)

Cell D (both wrong) – example 0.
Q: *John cuts grass at 2", grows 0.5"/mo, cuts at 4", costs $100/cut. Pays per year?*
Gold cot: `<<4-2=2>>` `<<2/.5=4>>` `<<12/4=3>>` `<<100*3=300>>` **Gold ans:** 300
Trace: k=0:`<<4-2=2>>` k=2:`*12=12>>` k=4:`<<12*100=1200>>` **Pred ans:** 100 (×)

PaRoutes. PaRoutes (Genheden and Bjerrum, 2022) is released as an open chemistry benchmark for retrosynthesis. We use the public training and evaluation splits as intended; we do not redistribute the underlying molecule data, only training and evaluation code that loads from the official release.

mech-USPTO-31K. The mech-USPTO-31K Data Descriptor (Chen et al., 2024b) provides arrow-pushing mechanism annotations for reactions drawn from USPTO-50K (Schneider et al., 2016). Both releases are intended as public chemistry-research benchmarks. Our use is consistent with this intended use, and our released code only loads from the official distribution.

MolPuzzle. MolPuzzle is a publicly released spectroscopy-to-structure benchmark distributed by its authors for research evaluation. We use it as intended (training and evaluating reasoning models that map NMR / IR / MS inputs to a SMILES output) and do not redistribute the underlying data.

Qwen3-0.6B backbone. Qwen3-0.6B (Qwen Team, 2025) is released under the Apache-2.0 license, which permits research use, modification, and redistribution of derivatives. Our finetuned anchor LM, question encoder, autoencoder, and suffix decoder are Apache-2.0-derivative checkpoints and will be released under the same terms.

Other tooling. RDKit (BSD-3-Clause), PEFT / LoRA (Hu et al., 2022) (Apache-2.0), and the PyTorch / Transformers stack are all used under their original open-source licenses and for their intended research purposes.

Artifacts we release. The training and inference code for ReasonFlow, the trained DiT denoisers, and the derived anchor / autoencoder / suffix-decoder checkpoints will be released under a research-only license consistent with the upstream Qwen3-0.6B Apache-2.0 license. We do not redistribute the raw upstream datasets; users must obtain those from the official sources cited above.

L Compute and Infrastructure

All experiments were run on single NVIDIA H200 GPUs (no multi-GPU or multi-node training). The total compute used for the experiments reported in this paper is on the order of 1,000 H200-GPU-hours, distributed across:

- Stage-1 autoencoder pretraining (per task / per pool-mode cell): $\sim$12 H200-hours each (see Section A).
- Anchor-LM finetuning (8,000 steps, batch 256): $\sim$12 H200-hours per task.
- Stage-2 anchored phase-2 diffusion training

Table 12: Retrosynthesis predictions where ReasonFlow's output is a valid alternative route that strict set-SMILES EM penalizes.

Target: `CCc1ccc(Nc2cc(=O)n(C)cc2C(N)=O)c(F)c1` **Gold reactants:** `C#C[Si](C)(C)C . Cn1cc(C(N)=O)c(Nc2ccc(I)cc2F)cc1=O` (Sonogashira-style with TMS-acetylene + iodoaniline) **Pred reactants:** `CCc1ccc(N)c(F)c1 . CI . COC(=O)c1cn(C)cc1=O . N` (Buchwald-Hartwig amination + N-methylation route)
Target: `CC(C)C(=O)N1CCC(CC(=O)Nc2ccc(-c3ccccc3)cc2)CC1` **Gold reactants:** `CC(C)(C)OC(=O)N1CCC(CC(=O)Nc2ccc(-c3ccccc3)cc2)CC1 . CC(C)C(=O)Cl` (Boc-deprotection + acylation) **Pred reactants:** `CC(C)C(=O)Cl . Nc1ccc(-c2ccccc2)cc1 . O=C(O)CC1CCNCC1` (direct amide coupling, free piperidine)
Target: `COC1CCC2(CC1)Cc1ccc(OCCCF)cc1C2=O` **Gold reactants:** `C=CC(=O)OC . CC(C)(C)[O-] . CI . O=C1CCc2ccc(O)cc21 . OCCCF` (multi-step: Michael + alkylation + Williamson) **Pred reactants:** `COC1CCC2(CC1)Cc1ccc(OCc3ccccc3)cc1C2=O . FCCCI` (benzyl-protected intermediate + Williamson with fluoroiodopropane)

Table 13: Three mech-USPTO-31K examples. The atom-mapping numbers are preserved verbatim; the model only needs to permute connectivity to form the product. Both strict no-CoT and ReasonFlow (mean_cot_only) emit the gold product byte-identically in $\sim 90\%$ of cases.

Reactants: `[CH2:1]([CH2:2][NH:3]...[n:12]1)[Cl:101]` **Gold product:** `[CH2:1]1[CH2:2][NH:3][C:4](=[O:5])[N:6]1[c:7]1...[n:12]1` **Pred (strict no-CoT):** byte-identical to gold (✓)
Reactants: `CC(C)(C)[O:202][C:201](=O)...[O:7].[NH2:8][CH2:9]...[c:16]1[NH2:17]` **Gold product:** `[CH3:1][C:2]([CH3:3])([CH3:4])[O:5][C:6](=[O:7])[NH:8][CH2:9]...[c:16]1[NH2:17]` (Boc protection) **Pred:** byte-identical to gold (✓)
Reactants: `[C:1]1(=[O:101])[c:2]2[cH:3][cH:4][c:5]([CH2:6]...[CH2:13]1.[NH2:14][OH:15].[H+:301]` **Gold product:** `[C:1]1(=[N:14][OH:15])[c:2]2[cH:3][cH:4][c:5](...)[CH2:13]1` (oxime formation) **Pred:** byte-identical to gold (✓)

(200,000 steps, batch 64, float32): $\sim$ 60–80 H200-hours per cell.

- Ablation grid (pool modes, projection dimensions, DiT sizes, M sweeps, recipe sweeps): the remaining majority of the compute budget.

Parameter counts. The backbone for every Qwen3-based trunk in the pipeline (anchor LM, question encoder f_Q, autoencoder encoder/decoder, suffix decoder) is Qwen3-0.6B ($\sim$ 0.6B parameters). The DiT denoiser configurations used in our ablations are DiT-S ($\sim$ 33M), DiT-B ($\sim$ 130M), DiT-L ($\sim$ 458M), and DiT-XL ($\sim$ 675M) parameters; the headline results use DiT-L. The latent dimension is $d = 256$, the LM hidden dimension is 1024, and we use LoRA r=32 adapters for the chemistry / logic transfer cascade and full fine-tuning for the GSM8K headline (see Section A). When LoRA is used, only $\sim$10M additional parameters per trunk are trained.

Software stack. PyTorch, HuggingFace Transformers, PEFT (for LoRA), and RDKit (for SMILES canonicalization). Mixed-precision training is used at Stage 1 but float32 is required at Stage 2 because the implied velocity field $u_\theta = (\hat{z}_1 - z_t)/(1-t)$ becomes numerically sensitive near $t \to 1$.

M Related Work

Chain-of-thought in autoregressive language models. Chain-of-thought prompting (Wei et al., 2022) and scratchpad training (Nye et al., 2021) established that interleaving intermediate reasoning steps before a final answer substantially improves accuracy on multi-step tasks. Subsequent work showed that step-level supervision (Lightman et al., 2023; Zelikman et al., 2022) and self-consistency (Wang et al., 2023) compound this gain. Latent-reasoning variants—Coconut (Hao et al., 2024) and Quiet-STaR (Zelikman et al., 2024)—place "thoughts" in a continuous space but still deploy them *autoregressively* at AR-token boundaries: each thought conditions the next token. Our reasoning instead lives along the continuous-time

axis of a flow; the trajectory is the reasoning, and there is no token-by-token generation of the answer.

Flow matching and diffusion. Flow matching (Lipman et al., 2023; Liu et al., 2023; Albergo et al., 2023) trains a vector field against linear-interpolant conditional paths in a simulation-free way; rectified flow (Liu et al., 2023) uses the straight-line interpolant we adopt. We use x0-prediction with sphere-normalized targets, a pattern shown to be numerically more stable than velocity prediction at $t \to 1$ (Karras et al., 2022). The denoiser is a DiT (Peebles and Xie, 2023) with PixArt-style cross-attention (Chen et al., 2024a) over a question encoder. Although technically distinct from score-based diffusion (Ho et al., 2020; Song et al., 2021), flow matching shares the structural property we exploit: the trajectory is parameterized by a continuous time variable along which intermediate states would otherwise be unsupervised. None of the prior flow / diffusion work, to our knowledge, anchors the intermediate clean target to a per-step semantic structure.

Diffusion for discrete reasoning, and Diffusion-of-Thought. Discrete diffusion (Lou et al., 2024; Campbell et al., 2022) applies diffusion-style denoising to discrete tokens directly. The closest neighbor to our setting is *Diffusion of Thought* (Ye et al., 2024) (Ye et al., 2024): they apply discrete-text diffusion to the reasoning tokens themselves, i.e., the diffusion model generates the reasoning chain (and answer) as a sequence of denoising steps over text. That is fundamentally different from our framing: there, reasoning is the *output* of a diffusion model over text; here, reasoning is *supervision* that shapes the intermediate clean target of a continuous flow whose output is the answer (decoded from a single endpoint latent). The two approaches address opposite ends of the same gap, and we view them as complementary.

Reasoning benchmarks and chemistry / logic transfer. GSM8K (Cobbe et al., 2021) provides our primary diagnostic via a formulaic re-derivation that yields token-level cot-step structure. Beyond arithmetic, we evaluate on PaRoutes (Genheden and Bjerrum, 2022) retrosynthesis, mech-USPTO-31K (Chen et al., 2024b) reaction-mechanism prediction, MolPuzzle (NMR/IR/MS-to-structure), and ProntoQA-harder (Saparov and He, 2023) multi-hop logical deduction. Per-task literature for retrosynthesis (template-based, template-free transformers (Schwaller et al., 2020), graph methods (Shi et al., 2020), search-based planners (Chen et al., 2020; Segler et al., 2018)), mechanism prediction (electron-flow models (Bradshaw et al., 2019), MechSMILES (Neukomm et al., 2025), Reactron (Chen et al., 2025), FlowER (Joung et al., 2025)), and logical proof (Tafjord et al., 2021; Creswell et al., 2023) is cited inline in the corresponding experimental sections.